(An Autonomous Institution, Affiliated to Anna University, Chennai)
Department of Artificial Intelligence and Data Science
V Semester B. Tech AIDS

AD23532 - Principles of Data Science

Academic Year: 2026-2027 **Batch:** 2024-2028

Ex 7: Model Planning and Building - Clustering

Name: Nischal **Register Number:** 241801188 **Date:** 09-10-2026

Dataset: Mall Customer Segmentation (Kaggle) **Tools:** Python, Pandas, NumPy, Matplotlib, Seaborn, scikit-learn

Learning Objective

To understand and implement the K-Means clustering algorithm for customer segmentation.

Problem Statement

A retail company wants to group its customers by Annual Income and Spending Score so that it can design personalised marketing campaigns. By identifying customers with similar purchasing behaviour, the company can improve customer satisfaction, optimise promotional offers and increase overall sales. The task is to analyse the customer dataset, apply K-Means clustering to find customer segments, visualise the clusters and recommend a marketing strategy for each segment.

Task 1: Load the Mall Customer Segmentation dataset

In [1]:

```
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import warnings; warnings.filterwarnings('ignore'); sns.set_theme(style='whitegrid')

df = pd.read_csv('Mall_Customers.csv')      # Mall Customer Segmentation dataset (Kaggle)
print("Dataset loaded successfully")
print("Shape:", df.shape)
```

Out [1]:

```
Dataset loaded successfully
Shape: (200, 5)
```

> **Observation:** The Mall Customers file has 200 customers and 5 columns: CustomerID, Gender, Age, Annual Income (k$) and Spending Score (1-100).

Task 2: Explore the dataset

In [2]:

```
print("First 5 records:"); print(df.head().to_string())
print("\nLast 5 records:"); print(df.tail().to_string())
print("\nShape:", df.shape)
print("\nData types:"); print(df.dtypes.to_string())
print("\nMissing values:"); print(df.isnull().sum().to_string())
print("\nDuplicate records:", df.duplicated().sum(), "| Duplicate CustomerID:", df['CustomerID'].duplicated().sum())
print("\nSummary statistics:"); print(df.describe().round(2).to_string())
print("\nGender counts:"); print(df['Gender'].value_counts().to_string())
```

Out [2]:

```
First 5 records:
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40

Last 5 records:
     CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
195         196  Female   35                 120                      79
196         197  Female   45                 126                      28
197         198    Male   32                 126                      74
198         199    Male   32                 137                      18
199         200    Male   30                 137                      83

Shape: (200, 5)

Data types:
```

```
CustomerID                int64
Gender                      str
Age                       int64
Annual Income (k$)        int64
Spending Score (1-100)    int64

Missing values:
CustomerID                0
Gender                    0
Age                       0
Annual Income (k$)        0
Spending Score (1-100)    0

Duplicate records: 0 | Duplicate CustomerID: 0

Summary statistics:
       CustomerID     Age  Annual Income (k$)  Spending Score (1-100)
count      200.00  200.00              200.00                  200.00
mean       100.50   38.85               60.56                   50.20
std         57.88   13.97               26.26                   25.82
min          1.00   18.00               15.00                    1.00
25%         50.75   28.75               41.50                   34.75
50%        100.50   36.00               61.50                   50.00
75%        150.25   49.00               78.00                   73.00
max        200.00   70.00              137.00                   99.00

Gender counts:
Gender
Female    112
Male       88
```

> **Observation:** The data is clean: no missing values and no duplicate records or customer IDs. Annual income ranges from 15 to 137 k$ (mean 60.56) and the spending score from 1 to 99 (mean 50.20). Both features are on a similar scale of about 100 points. The customers are 112 female and 88 male, with ages from 18 to 70 (mean 38.85).

Task 3: Select the features for clustering

In [3]:

```
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]
print("Selected features:", list(X.columns))
print("X shape:", X.shape)
```

Out [3]:

```
Selected features: ['Annual Income (k$)', 'Spending Score (1-100)']
X shape: (200, 2)
```

> **Observation:** Annual Income and Spending Score are used because they describe how much a customer can spend and how much the customer actually spends, which is what the marketing segments are based on. CustomerID is only an identifier, and Gender and Age are not needed for this task.

Task 4: Standardise the features with StandardScaler

In [4]:

```
sc = StandardScaler()
Xs = sc.fit_transform(X)
print(pd.DataFrame(Xs, columns=['Income_scaled', 'Spending_scaled'])
        .describe().loc[['mean', 'std', 'min', 'max']].round(3).to_string())
```

Out [4]:

```
      Income_scaled  Spending_scaled
mean         -0.000           -0.000
std           1.003            1.003
min          -1.739           -1.910
max           2.918            1.894
```

> **Observation:** After scaling, both features have mean 0 and standard deviation about 1 (1.003 here because pandas uses the sample standard deviation, while StandardScaler uses the population one). K-Means is distance based, so this stops a feature with a larger range from dominating the clusters.

Task 5: Visualise the selected features - scatter plot and pair plot

In [5]:

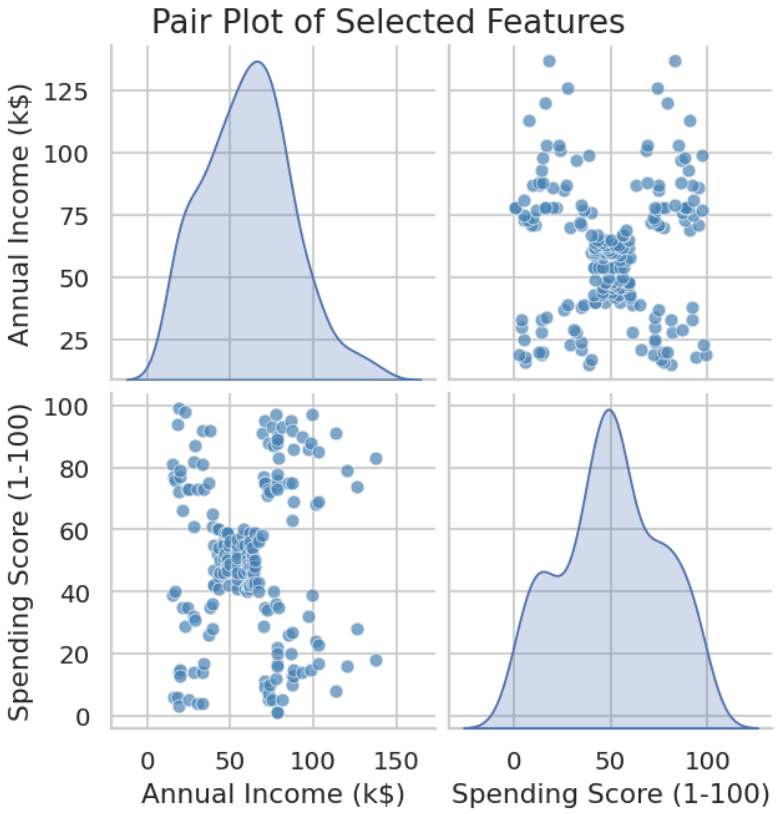
```
plt.figure(figsize=(7, 5))
sns.scatterplot(data=df, x='Annual Income (k$)', y='Spending Score (1-100)', s=50, color='steelblue')
plt.title('Spending Score vs Annual Income'); plt.tight_layout(); plt.show()

g = sns.pairplot(df[['Annual Income (k$)', 'Spending Score (1-100)']], diag_kind='kde',
                 plot_kws={'alpha': 0.7, 'color': 'steelblue'})
g.fig.suptitle('Pair Plot of Selected Features', y=1.02); plt.show()
print("Correlation (Income, Spending): %.3f" % X.corr().iloc[0, 1])
```

Out [5]:

```
Correlation (Income, Spending): 0.010
```





Observation: The correlation between income and spending score is almost zero (0.010), so one does not predict the other. The scatter plot shows distinct groups instead of a single trend: one dense group in the middle, and four groups in the corners (low income with low or high spending, high income with low or high spending). This shape suits K-Means clustering.

Task 6: Find the optimal number of clusters - Elbow Method

In [6]:

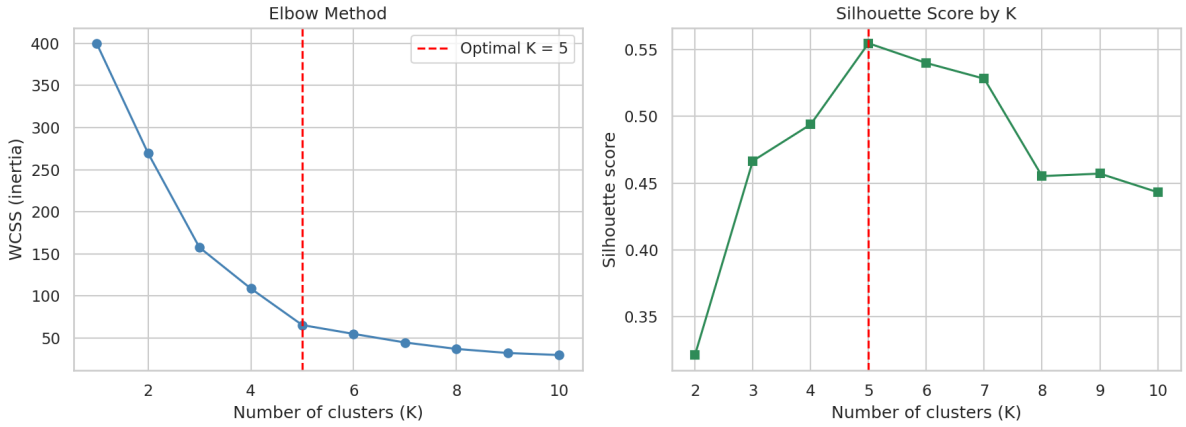
```
ks = range(1, 11)
wcss = [KMeans(n_clusters=k, random_state=42, n_init=10).fit(Xs).inertia_ for k in ks]
sil = {k: silhouette_score(Xs, KMeans(n_clusters=k, random_state=42, n_init=10).fit_predict(Xs))
       for k in range(2, 11)}
print("k   WCSS      Drop(%)     Silhouette")
for i, k in enumerate(ks):
    drop = '' if i == 0 else '%.1f' % ((wcss[i-1] - wcss[i]) / wcss[i-1] * 100)
    s = '' if k == 1 else '%.4f' % sil[k]
    print(f"{k:<3} {wcss[i]:8.2f}  {drop:>8}   {s:>8}")
print("\nOptimal K by elbow:", 5, "| K with highest silhouette:", max(sil, key=sil.get))

fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))
ax[0].plot(list(ks), wcss, 'o-'); ax[0].axvline(5, color='red', ls='--', label='Optimal K = 5')
ax[0].set_xlabel('Number of clusters (K)'); ax[0].set_ylabel('WCSS (inertia)'); ax[0].set_title('Elbow Method'); ax[0].legend()
ax[1].plot(list(sil), list(sil.values()), 's-', color='seagreen'); ax[1].axvline(5, color='red', ls='--')
ax[1].set_xlabel('Number of clusters (K)'); ax[1].set_ylabel('Silhouette score'); ax[1].set_title('Silhouette Score by K')
plt.tight_layout(); plt.show()
```

Out [6]:

k	WCSS	Drop(%)	Silhouette
1	400.00		
2	269.69	32.6	0.3213
3	157.70	41.5	0.4666
4	108.92	30.9	0.4939
5	65.57	39.8	0.5547
6	55.06	16.0	0.5399
7	44.86	18.5	0.5281
8	37.23	17.0	0.4552
9	32.39	13.0	0.4571
10	29.98	7.4	0.4432

```
Optimal K by elbow: 5 | K with highest silhouette: 5
```



Observation: WCSS falls steeply up to K = 5 and then flattens: the drop from K = 5 to K = 6 is only 16.0%, compared with 39.8% from K = 4 to K = 5. The elbow is therefore at K = 5. The silhouette score confirms it, as K = 5 has the highest value (0.5547). The optimal number of clusters is 5.

Task 7: Train K-Means with K = 5 and save the cluster labels

In [7]:

```
km = KMeans(n_clusters=5, random_state=42, n_init=10).fit(Xs)
df['Cluster'] = km.labels_                          # new column with the cluster label
df.to_csv('Mall_Customers_Clustered.csv', index=False)
chk = pd.read_csv('Mall_Customers_Clustered.csv')
print("Silhouette score for K = 5: %.4f" % silhouette_score(Xs, km.labels_))
print("Iterations:", km.n_iter_, "| Inertia: %.2f" % km.inertia_)
print("\nFile saved: Mall_Customers_Clustered.csv | shape:", chk.shape)
print(chk.head(8).to_string())
```

Out [7]:

```
Silhouette score for K = 5: 0.5547
Iterations: 4 | Inertia: 65.57

File saved: Mall_Customers_Clustered.csv | shape: (200, 6)
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)  Cluster
0           1    Male   19                  15                      39        4
1           2    Male   21                  15                      81        2
2           3  Female   20                  16                       6        4
3           4  Female   23                  16                      77        2
4           5  Female   31                  17                      40        4
5           6  Female   22                  17                      76        2
6           7  Female   35                  18                       6        4
7           8  Female   23                  18                      94        2
```

Observation: The model converged in 4 iterations with a silhouette score of 0.5547, which means the clusters are reasonably well separated. Each customer now has a Cluster label (0 to 4) in a new column, and the data was saved as Mall_Customers_Clustered.csv (200 rows, 6 columns).

Task 8: Visualise the clusters and the centroids

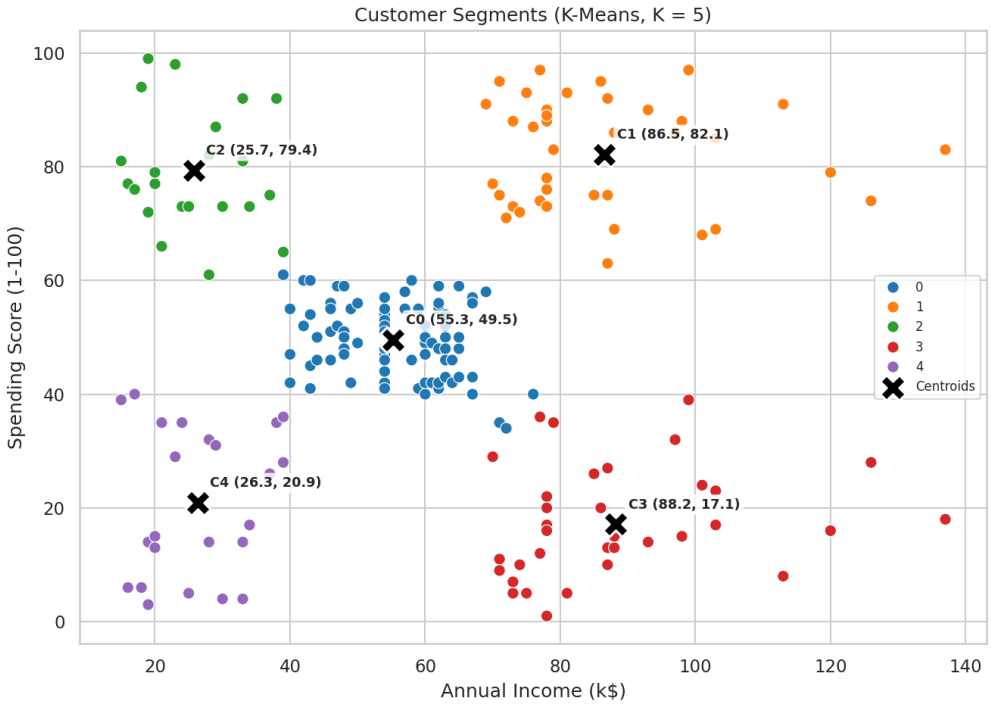
In [8]:

```
cent = pd.DataFrame(sc.inverse_transform(km.cluster_centers_),    # back to original units
                    columns=['Annual Income (k$)', 'Spending Score (1-100)'])
cent.index.name = 'Cluster'
plt.figure(figsize=(9, 6.5))
sns.scatterplot(data=df, x='Annual Income (k$)', y='Spending Score (1-100)',
                hue='Cluster', palette='tab10', s=55)
plt.scatter(cent.iloc[:, 0], cent.iloc[:, 1], c='black', marker='X', s=260,
            edgecolor='white', linewidth=1.5, label='Centroids', zorder=5)
for i, r in cent.iterrows():
    plt.annotate(f"C{i} ({r.iloc[0]:.1f}, {r.iloc[1]:.1f})", (r.iloc[0], r.iloc[1]),
                 textcoords='offset points', xytext=(8, 10), fontsize=8.5, fontweight='bold')
plt.title('Customer Segments (K-Means, K = 5)'); plt.legend(); plt.tight_layout(); plt.show()
print("Cluster centroids (original units):"); print(cent.round(2).to_string())
```

Out [8]:

```
Cluster centroids (original units):
```

```
         Annual Income (k$)  Spending Score (1-100)
Cluster
0                     55.30                   49.52
1                     86.54                   82.13
2                     25.73                   79.36
3                     88.20                   17.11
4                     26.30                   20.91
```



Observation: The plot shows five clearly separated segments, each drawn in its own colour, with the centroids marked by black crosses and labelled with their (income, spending) coordinates: C0 (55.30, 49.52), C1 (86.54, 82.13), C2 (25.73, 79.36), C3 (88.20, 17.11) and C4 (26.30, 20.91).

Task 9: Number of customers in each cluster

In [9]:

```
c = df['Cluster'].value_counts().sort_index()
print("Customers per cluster:")
print(pd.DataFrame({'Customers': c, 'Share (%)': (c / len(df) * 100).round(1)}).to_string())
print("Total:", c.sum())
```

Out [9]:

```
Customers per cluster:
         Customers  Share (%)
Cluster
0               81       40.5
1               39       19.5
2               22       11.0
3               35       17.5
4               23       11.5
Total: 200
```

Observation: Cluster 0 is the largest with 81 customers (40.5%). Clusters 1 and 3 have 39 and 35 customers. Clusters 2 and 4 are the smallest, with 22 and 23 customers. The five clusters add up to all 200 customers.

Task 10: Average Annual Income and Spending Score per cluster

In [10]:

```
prof = df.groupby('Cluster')[['Annual Income (k$)', 'Spending Score (1-100)']].mean().round(2)
prof['Customers'] = df['Cluster'].value_counts().sort_index()
prof['Avg Age'] = df.groupby('Cluster')['Age'].mean().round(1)
print(prof.to_string())
```

Out [10]:

```
         Annual Income (k$)  Spending Score (1-100)  Customers  Avg Age
Cluster
0                     55.30                   49.52         81     42.7
1                     86.54                   82.13         39     32.7
2                     25.73                   79.36         22     25.3
3                     88.20                   17.11         35     41.1
4                     26.30                   20.91         23     45.2
```

Observation: The averages equal the centroid coordinates. Cluster 1 has the highest income and spending, and cluster 4 the lowest of both. Cluster 2 earns little but spends a lot and is the youngest group (average age 25.3), whereas cluster 3 earns a lot but spends little. Cluster 0 sits in the middle on both. The older customers are in clusters 4 (45.2), 0 (42.7) and 3 (41.1).

Customer segments and recommended marketing strategies

Cluster	Segment	Profile (avg income, spending, customers)	Recommended marketing strategy
1	Premium customers	High income, high spending (86.5 k$, 82.1; 39)	Loyalty tiers, VIP and early access to new products, premium and personalised offers. Protect this group, as it brings the most revenue.
2	Young enthusiasts	Low income, high spending (25.7 k$, 79.4; 22)	Trend-led offers, student and youth discounts, instalment or reward-point schemes, social media campaigns. Encourage repeat visits without pushing them to overspend.
0	Standard customers	Average income, average spending (55.3 k$, 49.5; 81)	Seasonal sales, bundle deals and coupons that raise basket size. This is the largest group, so even a small uplift matters.
3	Untapped high earners	High income, low spending (88.2 k$, 17.1; 35)	Find out why they spend little (range, brands, store experience). Try premium and exclusive products, personalised recommendations and invitations to events.
4	Budget-conscious	Low income, low spending (26.3 k$, 20.9; 23)	Low-cost discount and clearance promotions, essentials and value packs. Keep marketing spend small for this group.

Result

Thus K-Means clustering was applied to the Mall Customer Segmentation dataset (200 customers). Annual Income and Spending Score were selected and standardised, and the Elbow Method (supported by the silhouette score of 0.5547) gave K = 5 as the optimal number of clusters. The cluster labels were added to the data and saved as Mall_Customers_Clustered.csv. The five segments have 81, 39, 22, 35 and 23 customers, with the centroids plotted and listed. Premium customers (high income, high spending) and untapped high earners (high income, low spending) are the key groups for targeted campaigns, and a marketing strategy was recommended for each segment.